

Moduli e Architetture

Marco Alberti



**Università
degli Studi
di Ferrara**

Ingegneria del Software Avanzata
CdLM in Ingegneria Informatica e dell'Automazione
A.A. 2021-2022

Ultima modifica: 8 aprile 2022

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright.
Ne sono vietati la riproduzione e il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore.

Sommario

- Cos'è un modulo
- Relazioni fra moduli
- Linguaggi per la rappresentazione della struttura modulare di un sistema
- Linee guida per la modularizzazione
- Architetture: tipiche suddivisioni modulari per classi di applicazioni

Sommario

1 Moduli

2 Architetture

Modulo software

Unità di codice che fornisce *servizi* o *risorse* ad altri moduli (o a entità esterne al sistema, come utenti) e utilizza servizi o risorse forniti da altri moduli.

Può consistere in:

- astrazione procedurale (funzione), incapsula algoritmo
- raccolte (librerie) di astrazioni procedurali
- pool di dati
- tipi di dato astratto (anche classi), rappresentazione dei dati nascosta
- librerie di tipi di dato

Tipicamente ogni linguaggio ha (almeno) una modalità per la rappresentazione dei moduli. Esempi:

- C: implicito (file header)
- Java: package (moduli logici), JAR (moduli fisici)
- JavaScript: diverse proposte (AMD, CommonJS, ES6)

Textual Design Notation (TDN)

Una notazione generale per la specifica di moduli e le loro relazioni.

In generale di un modulo è utile specificare

- identificatore
- posto occupato nell'eventuale composizione gerarchica del sistema
- quali servizi richiede (utilizza)
- quali servizi esporta (rende disponibili)

Nella specifica di modulo si omettono i dettagli dell'implementazione.

Relazione USES: dipendenza funzionale

Definizione

M_j **USES** M_k se e solo se M_j richiede M_k per il suo funzionamento

- M_j utilizza una risorsa o servizio (funzione di calcolo, struttura dati, etc.) resa disponibile da M_k .
- M_j è detto **client**, M_k è detto **server**.
- Relazione determinata staticamente dal progettista: sussiste anche se a tempo di esecuzione M_j non usa M_k .

Può essere rappresentata da un grafo diretto dove

- nodi rappresentano moduli
- un arco da M_j a M_k significa che M_j **USES** M_k

La **chiusura transitiva** di **USES** rappresenta dipendenza (se M_1 **USES** M_2 e M_2 **USES** M_3 , M_1 dipende da M_3).

Relazione IS-COMPONENT-OF: aggregazione

Definizione

M_j IS-COMPONENT-OF M_k se e solo se M_k è ottenuto aggregando vari moduli, uno dei quali è M_j .

Esempio

In Java, `java.util.function` IS-COMPONENT-OF `java.util`

Anche IS-COMPONENT-OF si può rappresentare con un grafo diretto dove

- nodi rappresentano moduli
- un arco da M_j a M_k significa che M_j IS-COMPONENT-OF M_k

Inversamente: IS-COMPOSED-OF

Definizione

M_j IS-COMPOSED-OF S se e solo se S è l'insieme di tutti i moduli M_k tali che M_k IS-COMPONENT-OF M_j .

Interfaccia

Un modulo esporta i servizi che rende disponibili ai suoi client

- L'insieme dei servizi esportati da un modulo è la sua **interfaccia**
- Il modo in cui l'interfaccia è realizzata (**implementazione**) è un *segreto* del modulo, cioè è nascosto ai client (o è *incapsulato* nel modulo)

Cosa esportare? Compromesso fra

- necessità di esportare più servizi per incrementare la funzionalità
- necessità di esportare meno servizi per mantenere semplice l'interfaccia
- E' meglio non esportare qualcosa che può cambiare (la modifica di una risorsa esportata è detta *breaking change*, in quanto causa malfunzionamenti dei moduli client)

Un modulo può importare anche solo alcuni servizi fra quelli esportati da un altro modulo.

Esempio TDN (1/2)

```
module X
  uses Y, Z
  exports var A : integer;
           type B : array (1..10) of real;
           procedure C (D: in out B; E: in integer; F: in real);
```

Here is an optional natural-language description of what A, B, and C actually are, along with possible constraints or properties that clients need to know; for example, we might specify that objects of type B sent to procedure C should be initialized by the client and should never contain all zeroes.

```
implementation
```

If needed, here are general comments about the rationale of the modularization, hints on the implementation, etc.
is composed of R, T

```
end X
```

Esempio TDN (2/2)

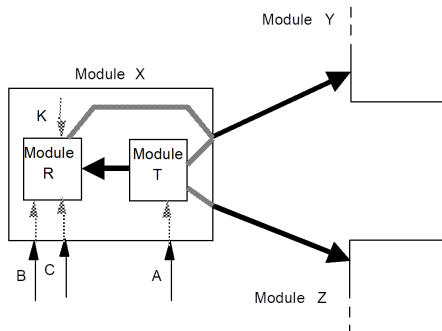
```
module R
  uses Y imports M, N
  exports var K : record . . . end;
      type B : array (1. .10) of real;
      procedure C (D: in out B; E: in integer; F: in real);
  implementation
    ...
end R
```

```
module T
  uses Y, Z, R
  exports var A : integer;
  implementation
    ...
end T
```

Graphical Design Notation (GDN)

Notazione grafica equivalente a TDN.

- Modulo rappresentato come rettangolo
- M_j **USES** M_k (eventualmente importando solo alcune risorse): freccia da M_j a M_k (eventualmente etichettata con le risorse importate)
- **IS-COMPONENT-OF**: inclusione spaziale



Linee guida per la modularizzazione

La scomposizione di un sistema in moduli non è banale.

Spesso la miglior suddivisione emerge durante la progettazione o anche durante lo sviluppo.

Alcune linee guida:

- Moduli piccoli, con responsabilità ben definite (riduce dipendenze funzionali)
- Alta coesione (dipendenze interne al modulo), basso accoppiamento (dipendenze fra moduli)
- Evitare dipendenze mutue (il grafo di **USES** dovrebbe essere aciclico): sistema più facile da implementare, verificare, mantenere
- Un modulo dovrebbe avere molti client (utile) e pochi server (indipendente)

Sommario

1 Moduli

2 Architetture

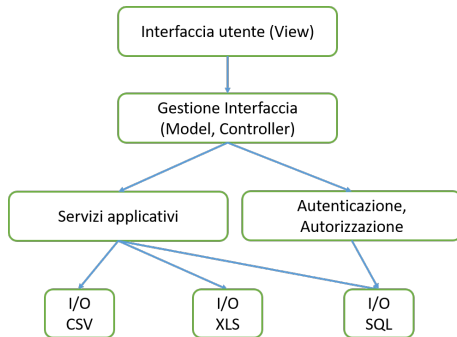
Architettura a strati

La relazione **USES** è tale che

- a ogni modulo può essere assegnato un intero detto *livello*
- ogni modulo **USES** solo moduli di livelli inferiori

I moduli dello stesso livello costituiscono uno *strato*.

Esempio:

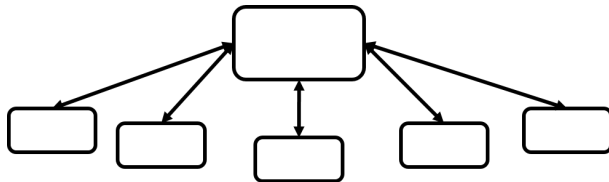


Pipeline



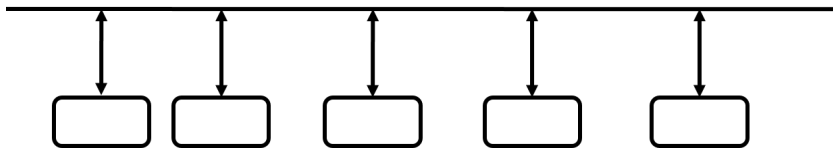
- Ogni sistema (modulo)
 - riceve input dal precedente
 - lo elabora
 - passa l'output al successivo
- Elaborazione di dati
- Detta anche pipe-and-filter (UNIX)

Blackboard



- Uno dei moduli (blackboard) funge da mezzo di comunicazione per gli altri
- Gli altri moduli, potenzialmente eterogenei, possono chiedere accedono alla blackboard in lettura e scrittura
- Adatto per sistemi distribuiti.

A eventi



- I componenti possono creare *eventi* o reagire a essi
- Modello publish-subscribe.
- Adatti a sistemi che devono attendere input o in cui le relazioni client-server non sono chiare

Architettura a due livelli

Sistema costituito da due moduli:

- ① client (ad esempio app nativa, web app): interfaccia con l'esterno (tipicamente utente), traduce le richieste dell'utente in interrogazioni e modifiche alla fonte dei dati, decodifica le risposte
- ② fonte dei dati (tipicamente RDBMS)

Vantaggi: semplicità

Svantaggi:

- Autorizzazione delegata alla fonte dei dati
- Necessario duplicare i servizi di applicazione per ogni client sviluppato

Architettura a tre livelli

Sistema costituito da tre moduli:

- ① client (ad esempio app nativa, web app): interfaccia con l'esterno (tipicamente utente)
- ② application server:
 - riceve richieste
 - le autorizza
 - interroga e modifica in modo opportuno la fonte dei dati
 - fornisce le risposte al client
- ③ fonte dei dati (tipicamente RDBMS)

Distribuzione fisica dei moduli

- Stesso thread: chiamata
- Diversi thread: memoria condivisa
- Processi diversi, stessa macchina: comunicazione inter-processo, filesystem
- Macchine diverse connesse da rete: remote procedure call, web services, HTTP
- Microservizi:
 - Insieme di server indipendenti, ognuno dei quali fornisce un singolo servizio
 - Insieme di client che accedono ai server di cui hanno bisogno
 - Facile da distribuire e scalare
 - Difficile gestire interazioni e garantire consistenza, transazionalità, assenza di corse critiche etc.

Scelta dell'architettura

In prima istanza: utilizzo di un'architettura che ha funzionato bene in casi simili.

Se non applicabile, scelta guidata da requisiti non funzionali:

- Prestazioni
 - Operazioni critiche confinate in un piccolo numero di sottosistemi
 - Minimizzazione della comunicazione fra sottosistemi (componenti grandi)
- Sicurezza: Architettura a strati, con le risorse più critiche all'interno
- Safety: Operazioni pericolose in un piccolo numero di sottosistemi per facilitare la verifica
- Disponibilità: Ridondanza per facilitare la sostituzione "a caldo"
- Mantenibilità: componenti piccoli, autonomi, facili da modificare